

ISEL | DEI | AMD / MLDM

Machine Learning and Data Mining

1. Context and objective (*just read; do nothing yet!*)

This MoP is inspired by the **Palmer Penguins** data. The original data were collected from 2007 to 2009 near Palmer Station, Antarctica, and include observations of Adélie, Chinstrap and Gentoo penguins.

The data used in this MoP are **adapted and enriched for teaching purposes**. They preserve several attributes from the Palmer Penguins data, while introducing a relational representation and additional information needed for the integration and completion tasks.

For the meaning of attributes inherited from the original data, consult Palmer Penguins raw-data documentation (https://allisonhorst.github.io/palmerpenguins/reference/penguins_raw.html) and published paper:

- Gorman, K. B., Williams, T. D., & Fraser, W. R. (2014), *Ecological Sexual Dimorphism and Environmental Variability within a Community of Antarctic Penguins (Genus Pygoscelis)*, PLoS ONE, 9(3), e90081: <https://doi.org/10.1371/journal.pone.0090081>

The original documentation can help clarify the meaning of attributes such as study, sample number, species, island, bill measurements, flipper length, body mass and sex.

Some attributes and structures used in this MoP are **course-specific extensions** and are therefore not described in the original Palmer Penguins documentation; their **meaning should be inferred from the provided schema and data**.

A PostgreSQL database contains an initial set of penguins and observations. Additionally, a comma-separated-values (CSV) file contains the data that was collected and must be integrated into that database source.

Some collected penguins may already be represented in the database, while some of their collected observations may still be new. After integration, part of the source also contains missing island information that must be completed using the available evidence.

The objective of this MoP is to move from separate and partially incomplete sources to one integrated and completed PostgreSQL data source.

Do not treat integration as simply appending rows. First understand the structure and contents of the sources; then determine what already exists, what is new, and what must be completed.

2. Working principles

While completing the activity:

- inspect the source material before manipulating it
- use the database structure and the meaning of the attributes to guide each join and filtering decision
- use expected-result checkpoints to detect mistakes, not as substitutes for understanding

- avoid changing rows or attributes that are not part of the current integration or completion task
- preserve the observation-level structure of the source
- keep all original `_mLDm_` files unchanged
- place each required artefact in the indicated folder, including the ER / EA model in `04_report`

Before considering the activity complete:

- make sure that the final evidence question in the last cell of notebook 02 has been answered
 - verify that all required student-created or student-modified artefacts are ready for submission
-

3. Course files and working copies

Files whose names begin with `_mLDm_` are original files provided by the course and must remain unchanged.

Only files that you are going to modify must be duplicated. For those files:

- duplicate the original `_mLDm_` file
 - remove the `_mLDm_` prefix from the duplicate filename
 - work only on the duplicate
 - keep the original course file unchanged
-

3.1. Prepare the working copies

In this MoP, the following files in `02_code` must be duplicated and the `_mLDm_` prefix removed:

- `_mLDm_00_penguins_db_boot.py`
to `00_penguins_db_boot.py`
- `_mLDm_01_penguins_integration.ipynb`
to `01_penguins_integration.ipynb`
- `_mLDm_02_penguins_completion.ipynb`
to `02_penguins_completion.ipynb`

Keep the original `_mLDm_` files unchanged and work only on the corresponding copies.

3.2. Select the container runtime

Open:

`02_code/00_penguins_db_boot.py`

Near the beginning of the file, select the container runtime that you are using:

```
RUNTIME = "podman"
# RUNTIME = "docker"
```

If you use **Podman**, keep the configuration above.

If you use **Docker**, comment the Podman line and uncomment the Docker line:

```
# RUNTIME = "podman"
RUNTIME = "docker"
```

Exactly one of these two assignments should be active.

This choice must correspond to the container environment that you configured and tested in the previous MoP.

4. Inspect the source material before running the notebooks

Before executing the notebooks, inspect the three course-provided data-source files in `01_input`.

4.1. Inspect the database schema

Open:

```
_mLDM_01_penguins_schema.sql
```

Read the SQL schema and reconstruct the conceptual structure represented by it.

Create an **Entity–Relationship (ER)** model or, in Portuguese, "modelo **Entidade-Associação (EA)**", showing:

- the entities,
- their attributes, including primary keys,
- the relationships between entities,
- the cardinality of each relationship,
- the optionality of each relationship.

Preferably, use Crow's Foot notation, or, in Portuguese, "notação pé-de-galo", a widely used notation for data modelling, as it provides a more compact representation of cardinality and optionality. The well-known Chen notation is also valid, but its use of double lines for mandatory (total) participation can make larger diagrams visually heavier.

Use any diagramming tool you prefer; e.g., diagrams.net (<https://www.drawio.com>) is a convenient option for creating the ER model. In diagrams.net, you may enable the "Entity Relation" shape library (via More Shapes... \ Software \ Entity Relation \ Apply) to access ER elements and Crow's Foot connectors.

Save or export the resulting ER model as a PDF file named `penguins_ER_model.pdf` and place it in `04_report`. You may also keep the editable source file used by your diagramming tool.

The purpose of this task is to develop a conceptual understanding of the data model before manipulating its contents.

4.2. Inspect the initial database data

Open:

`_mLDm_02_penguins_data_db.sql`

Inspect the essential data that will be loaded into the initial PostgreSQL database. In particular, identify:

- which tables receive data
- the kind of information stored in those tables
- how the inserted values relate to the schema you have just analysed
- any aspects that may later matter when integrating new data

Do not modify this file.

4.3. Inspect the collected CSV data

Open:

`_mLDm_00_penguins_data_collected.csv`

Inspect its structure and contents before importing it. Identify:

- what one row represents
- the available attributes
- which attributes can be related to the relational schema
- whether missing values are present
- collected data that may overlap with information already stored in the database

Do not modify this file.

At this stage you are not expected to solve the integration problem. The goal is to understand the sources well enough to make informed decisions in the notebooks.

5. Files and execution order

5.1. Input files

The `01_input` folder contains:

- `_mLDm_00_penguins_data_collected.csv`
- `_mLDm_01_penguins_schema.sql`
- `_mLDm_02_penguins_data_db.sql`
- `_mLDm_xx_submission_requirements.txt`

The submission requirements file is provided by the course and must not be modified. It is used by the submission tool to validate the required artefacts.

5.2. Code files

The 02_code folder contains:

- `_mLDM_00_penguins_db_boot.py`
- `_mLDM_01_penguins_integration.ipynb`
- `_mLDM_02_penguins_completion.ipynb`
- `_mLDM_xx_build_submission_zip.py`

After creating the working copies and configuring the container runtime, execute the notebooks in this order:

1. `01_penguins_integration.ipynb`
2. `02_penguins_completion.ipynb`

Notebook 01 runs the bootstrap script (`00_penguins_db_boot.py`) automatically and recreates the initial PostgreSQL database. The bootstrap script therefore does not need to be executed separately; it only needs to be configured with the appropriate container runtime.

The notebooks contain expected-result checkpoints; use them to validate intermediate steps.

6. Part A — Data integration

Open:

`01_penguins_integration.ipynb`

The notebook imports the collected CSV data into a PostgreSQL staging table and provides the technical setup required for the integration.

Complete the TODO sections so that you can:

- distinguish collected observations from distinct penguins
- compare collected penguins with penguins already stored in the database
- inspect overlapping information and check its consistency
- identify penguins that still need to be added
- identify observations that still need to be added

Pay particular attention to the distinction between the two database concepts:

- PENGUIN represents the penguin entity and its stable attributes
- OBSERVATION represents an individual observation of a penguin

A penguin can therefore already exist in PENGUIN while a collected observation of that penguin is still absent from OBSERVATION.

Use the validation cells to check the state of the integrated database before continuing.

7. Part B — Source completion

Open:

`02_penguins_completion.ipynb`

only after the integration notebook has been completed successfully.

Notice that some observations have missing island information but include geographic coordinates. For each incomplete observation, the island is inferred by comparing its geographic position with the reference centroid of each known island. The distance to each island centroid is computed using the Haversine formula, and the island associated with the smallest distance is selected.

Complete the TODO sections so that you can:

- compute the distance from each incomplete observation to each island reference
- obtain the minimum distance for each observation
- identify the corresponding nearest island
- update only the observations with missing island information

Use the final validation query to confirm that the missing island values were completed without changing the number of observations.

The result of this notebook is not a new dataset file. Its main output is the persistent state of the PostgreSQL database, now containing the integrated and completed data source that will be used in the next MoP.

8. First visual checkpoint with Orange Data Mining (DM)

After completing the source integration and completion steps, use **Orange DM** for a first visual inspection of the original collected CSV data.

This checkpoint deliberately uses the collected CSV as a simple introduction to Orange DM. It is not intended to validate the completed PostgreSQL source. The goal is just to become familiar with visual data exploration and with the idea that attributes available in a data source may play different roles in an analytical view. The integrated source will be explored more systematically in the next part of the activity.

Follow the instructions near the end of `02_penguins_completion.ipynb` to:

- install Orange DM as a standalone application, if necessary
 - load the collected CSV data
 - use `Select Columns` to distinguish target, features, meta attributes, and ignored attributes
 - inspect the data using simple visualisation widgets
 - save the Orange workflow in `02_code`
 - save the requested visual evidence in `04_report`
-

9. Required final evidence answer

Do not finish the activity before answering the final evidence question.

The **last cell** of `02_penguins_completion.ipynb` contains a **required final evidence question**.

You must answer all parts of that question before submission.

The purpose of the final answer is to connect the operations performed in notebooks 01 and 02 with concrete evidence obtained during their execution. Keep the answer concise and support it with results from the notebooks.

Before submitting, explicitly verify that the final evidence cell has been answered.

10. Submission checklist

Before submitting, verify that you have:

- `02_code/00_penguins_db_boot.py`
- `02_code/01_penguins_integration.ipynb`
- `02_code/02_penguins_completion.ipynb`
- `02_code/penguins_visual_check.ows`
- `04_report/penguins_ER_model.pdf`
- `04_report/penguins_orange_evidence.png`
- the final evidence question answered in the last cell of `02_penguins_completion.ipynb`

Use `_mLDm_xx_build_submission_zip.py`, as in the previous MoP, to validate the required artefacts and create the ZIP file for submission.

The tool uses `01_input/_mLDm_xx_submission_requirements.txt` to determine which artefacts are required. It also creates `04_report/submission_check.txt`.

If a required artefact is missing, the validation fails and the submission ZIP is not created.

No generated dataset file is expected in `03_output` in this part of the activity.

Before uploading the ZIP to Moodle, check that `submission_check.txt` reports a successful validation and inspect the ZIP contents.